

스마트 반려동물 자동 급식기 개발보고서

이름 : 이세연

1

개발 목표와 특징

- 개발하고자 하는 제안시스템의 특징을 반영한 개발목표를 기술

아두이노 UNO3를 사용하여 스마트 반려동물 자동 급식기를 개발하는 것을 목표로 함. 이 작품은 정해진 급식 시간에 노래 소리가 나오면서 자동으로 반려동물의 접근을 감지하여 사료를 제공함. 또한 사료통의 내부 습도를 인식하여 LED를 통해 사료의 종류를 나타내고, LCD를 통해 다음 배식까지 남은 시간을 확인할 수 있음. 또한 수동 버튼으로 수동으로 사료 배급도 가능함

2

개발 필요성

- 왜 본 시스템 개발이 필요한지 기술적 환경 및 수요 배경을 중심으로 기술

통계청 자료를 바탕으로 2023년 기준 1인 가구는 전체 가구의 35.5%를 차지하며, 점점 증가하는 추세를 보이고 있음. 또한 농림축산식품부의 2024 동물 복지에 대한 국민 의식 조사에 따르면 국내 반려동물 현황은 2024년 기준 286%가 반려동물을 양육하고 있음. 1인가구의 10가구중 1가구는 반려동물을 보유하고 있다는 통계청의 결과에 따르면, 바쁜 일상을 보내는 반려동물 보호자들은 규칙적인 급식시간을 지키기 어려운 경우가 많고, 출장이나 여행 등으로 집을 비울 때 반려동물의 식사관리가 큰 문제가 됨. 이러한 문제의 해결 방안으로 자동급식기의 필요성이 증가하고 있음.

3

H/W 구현

- H/W를 어떻게 구현했는지를 그림, 사진 등으로 설명, 사용 센서, 부품들을 나열

하드웨어 전체 모습이다. 맨 위에는 수동 급식을 위한 텍스트 스위치가 달려있으며, 그 아래로 LCD가 위치하고, 건식, 습식 LED를 차례대로 배치하였다. 그 아래 폼보드에 구멍을 뚫어 밥그릇이 들어가는 공간을 마련하였고, 사료가 옆으로 새는 확률을 줄이고자 옆에 가림막도 달았다. 맨 아래에는 초음파센서가 부착되어있고, 초음파센서가 위치한 곳 안쪽으로 아두이노와 회로들을 담았다. 각 센서들을 센서들에 부착하기 위해 위쪽으로 구멍을 뚫어서 연장된 케이블들로 부착하였다.



온습도센서는 사료통의 역할을 하는 페트병에 부착하였고 나머지는 폼보드에 구멍을 뚫어 끼워넣어 고정하였다.



사료통이기에 뚜껑이 꼭 필요하다고 생각하여 원형의 형태로 구멍을 뚫고, 더 큰 원을 하나 잘라서 면봉과 글루건을 이용해 돌아가는 뚜껑을 제작하였다.



사료통의 내부는 서보모터에 연결된 납작한 판으로 구멍을 가렸다 열어주는 방식으로 사료를 배출한다. 경사로를 만들어 사료가 아래로 흐를 수 있도록 설계하고, 옆으로 끼지 않도록 가림막을 세워주었다.



4

S/W 구현

□ S/W를 어떻게 구현했는지를 설명

1. 자동 급식 기능

[밥 먹을 시간이 되었을 때]

< 강아지 접근 시 >

=> 정해진 시간 → 부저 알림 → 감지창 시작 → 강아지 접근 (초음파센서)
→ 시리얼 모니터에 밥을 준다는 말 출력 및 feed 함수로 서보모터 구동

< 강아지 미 접근 시 >

=> 정해진 시간 → 부저 알림 → 감지창 시작 → 강아지 미접근 (초음파센서)
→ 강아지가 밥을 먹으러 오지 않았어요 → 밥을 줄까요? (예 / 아니요)

[평상시]

강아지 접근 (초음파센서) → 강아지 접근 알림 → 밥을 줄까요? (예 / 아니요)

2. 사료 모니터링 기능

- DHT센서를 통한 사료통 내부 습도 측정 → 시리얼 모니터에 현재 습도 값 출력
- 습도 70% 미만: 건식 LED ON, 습식 LED OFF
- 습도 70% 이상: 건식 LED OFF, 습식 LED ON

3. 알림 기능

- 자동 급식 시간 마다 부저로 멜로디 알림
- playMelody() 함수에서 '작은별' 멜로디를 부저(BUZZER_PIN)로 재생

4. 남은 시간 확인

- 다음 배식까지 남은 시간 LCD에 표시

5

구현 소감

□ 개발 과정에 있었던 일과 느낌을 소개(개인별)

처음 개발을 시작할 때는 어디서부터 시작을 해야할지가 막막했다.

하드웨어를 만들기 전에 소프트웨어 코드를 짜면서

수업시간에 배웠던 코드들을 다시 보면서 어떤 코드를 써야할지 생각했다.

배운 코드들을 수정하여 실제로 적용해보고 각각 센서 하나씩이 아닌 LCD부터, 초

음파센서, 온습도센서, LED, 탭트스위치까지 모든게 한번에 실행되는 과정이 너무 신기하고 재밌었다. 이 개발작품은 상황에 따라 달라지는 센서들이 있어서 if문 활용을 많이 해야하는데 상태변수를 이용하거나, if문 조건 같은 것들을 ai의 도움을 받았다. 단순 센서들의 작동들은 다 정상적이었지만, 각종 조건들이 달리니 꼬이기도 하고 이상하게 실행되기도 해서 많이 어려웠던 것 같다. 하드웨어를 만들때에도 설계를 해본적이 없어서 엉성하게 시작하게 되어서 아쉬움이 있었다. 모터를 잘못 부착하여 떼고 다시 부착하거나, 폼보드에 구멍을 뚫지 않고 먼저 붙여버려서 커다란 하드웨어를 들고 한쪽면에 구멍을 뚫는 등 여러 가지 시행착오를 겪으며 만들었다. 이런 경험을 하고나니 다음에 또 이런 제작을 하게 되면 길이나 각도를 더 세심하게 정하고, 설계를 더 완벽히 할 수 있겠다는 생각이 들었다. 막막했지만 다 완성하니 엄청 뿌듯하고 해냈다는 기분에 행복했다.

6

향후 보완 사항

- ☐ 향후 보완이 필요하다고 생각되는 사항 기술

향후 보완이 필요하다고 생각되는 사항은 사료가 떨어지는 경사면의 각도이다. 설계시에는 사료가 잘 떨어지게 하기 위해서 어느정도의 각도를 세워서 설계하였지만, 다 만들고 시연을 해보니 경사가 높아 사료가 빠른 속도로 떨어지면서 밥그릇 밖으로 튈는 현상이 생겼다. 향후 각도를 좀 더 완만하게 제작하여 사료가 떨어지는 속도를 늦추면 튈는 현상이 사라질 것이라고 생각한다.

7

참고 문헌

- ☐ 개발 과정에 참고한 도서, 사이트 등을 기술

8

스케치 소스 코드

- ☐ 스케치 코드를 첨부하세요. 가져온 소스는 표시하세요.
텍스트로 표기 , 센서 사용 코드 가져왔던가, ai가 짜준게잇는지
내가 만든게 원지 표기

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <Servo.h>
#include <DHT.h>

#define TRIG_PIN 4
#define ECHO_PIN 5
```

```
#define SERVO_PIN 9
#define BUZZER_PIN 8
#define SWITCH_PIN 7
```

```
#define DHT_PIN 6
#define DHT_TYPE DHT11
```

```
#define LED_DRY 11
#define LED_WET 10
```

```
Servo myServo;
LiquidCrystal_I2C lcd(0x27, 16, 2);
DHT dht(DHT_PIN, DHT_TYPE);
```

```
const int openAngle = 90;
const int closeAngle = 180;
const int detectDistance = 20;
const unsigned long openDuration = 2000;
const unsigned long melodyInterval = 120000UL;
```

```
const unsigned long detectWindow = 20000UL;    -> UL이라는 단위 Ai 제공
```

```
unsigned long lastMelodyTime = 0;
unsigned long detectWindowStart = 0;
bool detectWindowActive = false;
bool prevDetected = false;
bool feedingDoneInWindow = false;
unsigned long lastDhtUpdate = 0;
bool waitingForManualFeed = false;
```

```
void setup() {
  Serial.begin(9600);
  Serial.println("-----");
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(BUZZER_PIN, OUTPUT);
  pinMode(SWITCH_PIN, INPUT_PULLUP);
  pinMode(LED_DRY, OUTPUT);
```

```
pinMode(LED_WET, OUTPUT);
myServo.attach(SERVO_PIN);
myServo.write(closeAngle);
```

```
lcd.init();
lcd.backlight();
lcd.setCursor(0, 0);
lcd.print("Smart Feeder");
```

```
dht.begin();
```

```
playMelody();
startDetectWindow();
}
```

```
void loop() {
  unsigned long now = millis(); -> 현재 시간 읽는 문장 AI 제공
  /
  if (!detectWindowActive) {
    displayTimeToNextFeed(now);
  } else {
    lcd.setCursor(0, 1);
    lcd.print("          "); -> 전에 출력된 거 지우는 코드 Ai제공
  }
}
```

```
if (now - lastDhtUpdate > 2000) { -> 2초마다 센서가 돌도록 Ai 제공
  updateDhtAndLed();
  lastDhtUpdate = now;
}
```

```
if (digitalRead(SWITCH_PIN) == LOW) {
  Serial.println("수동 급식 버튼이 눌렸습니다. 밥을 줍니다.");
  feed();
  lastMelodyTime = millis();
  delay(1000);
}
```

```
if (detectWindowActive) {
```

```

int distance = measureDistance();
if (!feedingDoneInWindow && distance > 0 && distance < detectDistance &&
    !prevDetected) {
Serial.println("강아지가 접근했습니다! 밥을 줍니다.");
feed();
feedingDoneInWindow = true;
prevDetected = true;
lastMelodyTime = millis();
}
if (distance >= detectDistance) prevDetected = false;
if (now - detectWindowStart >= detectWindow) {
    detectWindowActive = false;
    if (!feedingDoneInWindow) {
        Serial.println("강아지가 밥을 먹으러 오지 않았어요.");
        Serial.println("밥을 줄까요? (예/아니요)");
        waitingForManualFeed = true;
        lastMelodyTime = now;
    }
    feedingDoneInWindow = false;
}
delay(100);
return;
}    => 초음파센서 감지창 부분 Ai 도움

```

```

// 감지창이 아닐 때, 2분이 지나면 멜로디/감지창 시작
if (!detectWindowActive && (now - lastMelodyTime >= melodyInterval)) {
playMelody();
startDetectWindow();
}

```

```

int distance = measureDistance();
if (!detectWindowActive && distance > 0 && distance < detectDistance &&
    !prevDetected && !waitingForManualFeed) {
Serial.println("강아지가 접근했어요 !");
Serial.println("밥을 줄까요? (예/아니요) 입력 후 엔터");
waitingForManualFeed = true;
prevDetected = true;
}

```



```
}
```

```
if (distance >= detectDistance) prevDetected = false; => 평상시 초음파 감지코드 Ai  
제공
```

```
if (waitingForManualFeed && Serial.available()) {  
  String answer = Serial.readStringUntil('\n');  
  answer.trim();  
  if (answer == "예") {  
    Serial.println("밥을 줍니다.");  
    feed();  
    waitingForManualFeed = false;  
    lastMelodyTime = millis();  
  } else if (answer == "아니요") {  
    Serial.println("밥을 주지 않습니다.");  
    waitingForManualFeed = false;  
  } else {  
    Serial.println("예 또는 아니요로 입력해주세요.");  
  }  
}
```

```
delay(50);  
}
```

```
void displayTimeToNextFeed(unsigned long now) {  
  unsigned long elapsed = now - lastMelodyTime;  
  unsigned long remain;  
  if (elapsed < melodyInterval) {  
    remain = melodyInterval - elapsed;  
  } else {  
    remain = 0;  
  }  
  unsigned long seconds = remain / 1000;  
  unsigned int h = seconds / 3600;  
  unsigned int m = (seconds % 3600) / 60;  
  unsigned int s = seconds % 60;
```

```
  lcd.setCursor(0, 1);
```

```
lcd.print("Next:");  
lcd.print(h);  
lcd.print("h ");  
if (m < 10) lcd.print('0');  
lcd.print(m);  
lcd.print("m ");  
if (s < 10) lcd.print('0');  
lcd.print(s);  
lcd.print("s ");  
lcd.print("    ");  
}
```

```
void updateDhtAndLed() {  
float humidity = dht.readHumidity();  
Serial.print("현재 습도: ");  
Serial.print(humidity);  
Serial.println("%");  
if (isnan(humidity)) {  
digitalWrite(LED_DRY, LOW);  
digitalWrite(LED_WET, LOW);  
return;  
}  
if (humidity < 70 ) {  
digitalWrite(LED_DRY, HIGH);  
digitalWrite(LED_WET, LOW);  
} else {  
digitalWrite(LED_DRY, LOW);  
digitalWrite(LED_WET, HIGH);  
}  
}
```

```
void startDetectWindow() {  
detectWindowActive = true;  
detectWindowStart = millis();  
feedingDoneInWindow = false;  
prevDetected = false;  
Serial.println("20초 자동급식 감지창이 시작되었습니다.");
```

} => 감지창 함수 Ai 도움

```
int measureDistance() {  
  digitalWrite(TRIG_PIN, LOW);  
  delayMicroseconds(2);  
  digitalWrite(TRIG_PIN, HIGH);  
  delayMicroseconds(10);  
  digitalWrite(TRIG_PIN, LOW);
```

```
  long duration = pulseIn(ECHO_PIN, HIGH, 30000);  
  int distance = duration * 0.034 / 2;  
  return distance;  
} => 초음파센서 거리측정 Ai 도움
```

```
void feed() {  
  myServo.write(openAngle);  
  delay(openDuration);  
  myServo.write(closeAngle);  
}
```

```
void playMelody() {  
  int melody[] = {  
    262, 262, 392, 392, 440, 440, 392,  
    349, 349, 330, 330, 294, 294, 262  
  };  
  int noteDurations[] = {  
    400, 400, 400, 400, 400, 400, 800,  
    400, 400, 400, 400, 400, 400, 800  
  };  
  => 작은별 멜로디 인터넷 검색
```

```
  for (int thisNote = 0; thisNote < 14; thisNote++) {  
    tone(BUZZER_PIN, melody[thisNote], noteDurations[thisNote]);  
    delay(noteDurations[thisNote] * 1.3);  
  }  
  noTone(BUZZER_PIN);  
} =>for문을 이용하여 멜로디 출력 인터넷 검색
```